

전체 작업 흐름 한눈에 보기

[최초 1 회]

SSH 설정 → git clone → cd 저장소 → git fetch → 담당 feature 브랜치 진입

[매 작업 시작]

git switch 담당브랜치 → git pull origin 담당브랜치 → 작업

[작업 저장]

git status → git add . → git commit -m "..." → git push origin 담당브랜치

[기능 완료]

GitHub Pull Request: feature/... → develop → 리뷰/확인 → merge

[최종 통합]

develop 검증 완료 → main 으로 PR → merge

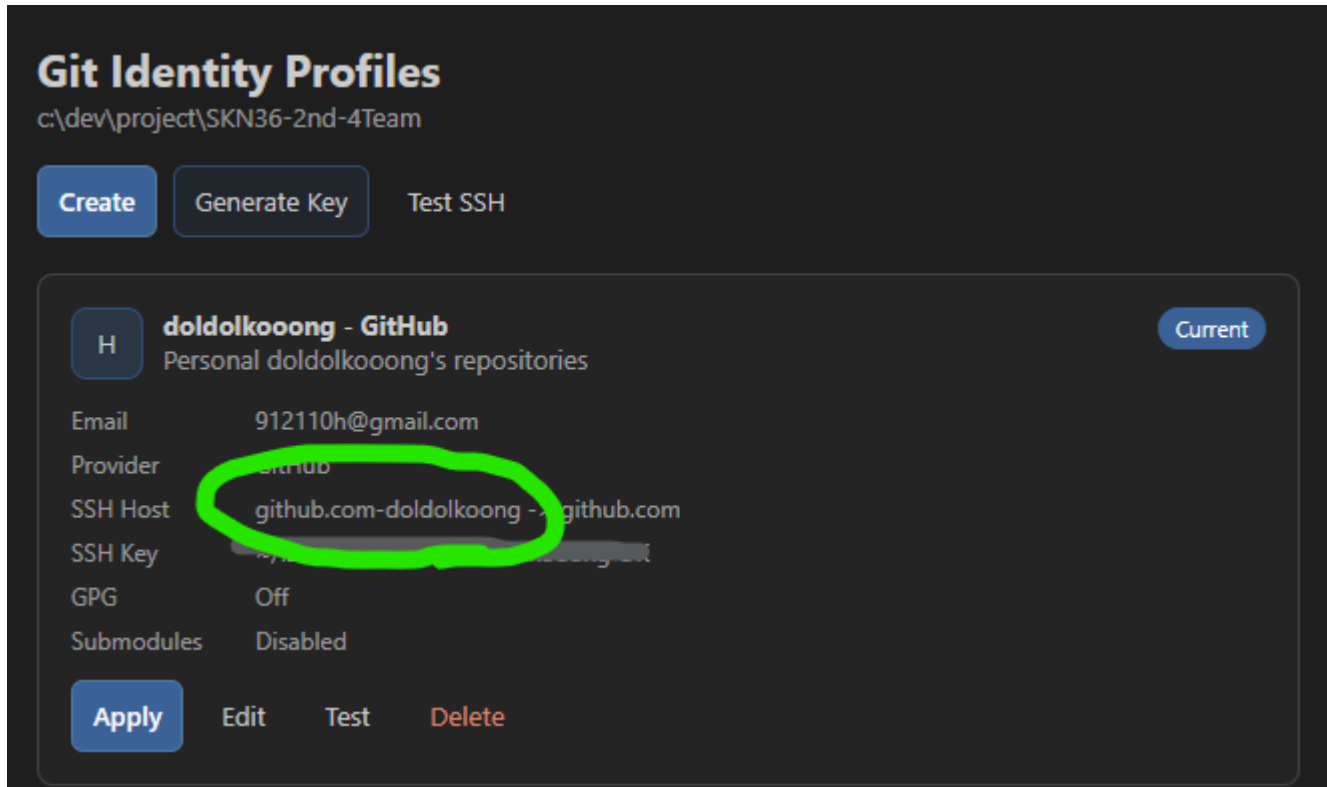
목차

1. SSH Clone
2. 브랜치 구조
3. 브랜치 사용 규칙
4. 담당 브랜치 진입
5. Pull / Add / Commit / Push
6. 하루 작업 예시
7. Pull Request
8. develop 내용 반영
9. 오류 해결
10. 최종 체크리스트

1. SSH 방식으로 GitHub 저장소 Clone 하기

1-1. SSH URL 로 저장소 Clone

GitHub 저장소 페이지에서 Code 버튼을 누른 뒤 SSH 탭을 선택하고 주소를 복사합니다. 주소 형태는 다음과 같습니다.



[git@github.com](https://github.com)-본인 alias:SKNETWORKS-FAMILY-AICAMP/SKN36-2nd-4Team.git

1-2. Clone 직후 원격 브랜치 확인

```
git fetch --all --prune
git branch -a
```

아래와 비슷하게 origin/main, origin/develop, origin/feature/...가 보이면 정상

```
remotes/origin/main
remotes/origin/develop
remotes/origin/feature/data-collection
```

remotes/origin/feature/data-analysis

remotes/origin/feature/churn-model

remotes/origin/feature/dashboard

2. 우리 프로젝트의 GitHub 브랜치 구조

```
main
├── develop
│   ├── feature/data-collection
│   ├── feature/data-analysis
│   ├── feature/churn-model
│   └── feature/dashboard
```

브랜치	역할	직접 작업 여부
main	발표·배포 가능한 최종 안정 버전	원칙적으로 직접 작업하지 않음
develop	각 feature 결과를 모아 통합 테스트하는 브랜치	통합/검증 목적
feature/data-collection	데이터 수집, API, 원천 데이터 생성	담당자 작업
feature/data-analysis	전처리, EDA, 파생변수 생성	담당자 작업
feature/churn-model	모델 학습, 평가, 예측 로직	담당자 작업
feature/dashboard	Streamlit/UI/대시보드 구현	담당자 작업

각 팀원은 자신이 맡은 feature 브랜치에서 작업합니다. 브랜치는 사람 이름 기준이 아니라 기능 기준으로 나뉘기 때문에 담당자가 바뀌더라도 구조를 그대로 유지할 수 있음(웬만해선 안바뀜)

중간 내용 계속 바뀌고 하는 거면 브랜치 더 나눠줄 수 있음

Dashboard 는 저랑 반석님이 할 거니까 더 나눠줄게요)

3. 브랜치 사용 규칙

- main 에서 코드를 직접 수정하거나 push XXX
- develop 도 일반 개발용이 아니라 통합용으로 사용하니까 사용 금지
- 실제 작업은 반드시 본인의 담당 feature 브랜치에서
- 작업 시작 전에 git pull 로 담당 브랜치 최신 상태를 먼저 받기
- 의미 있는 작업 단위로 commit, 하루치 전체를 한 번에 뭉쳐 commit XXX
- 작업 후에는 자신의 feature 브랜치로 push
- 기능이 완료되면 feature → develop 방향으로 Pull Request 를 생성
- develop 에서 전체 기능을 확인한 뒤 최종적으로 develop → main PR 을 진행

4. 담당 브랜치로 들어가는 방법

4-1. 처음 clone 한 팀원

clone 직후에는 보통 main 에 있습니다. 먼저 원격 브랜치 정보를 최신화

```
git fetch --all --prune  
git branch -a
```

그 다음 본인이 담당할 브랜치로 로컬 추적 브랜치를 생성하며 이동

```
git switch -c feature/data-collection --track origin/feature/data-collection
```

다른 담당자는 브랜치 이름만 바꿈

```
# 데이터 분석 담당  
git switch -c feature/data-analysis --track origin/feature/data-analysis  
  
# 모델링 담당  
git switch -c feature/churn-model --track origin/feature/churn-model  
  
# 대시보드 담당  
git switch -c feature/dashboard --track origin/feature/dashboard
```

4-2. 이미 로컬 브랜치를 만들어 둔 팀원

이미 한 번 해당 브랜치에 들어간 적이 있다면 -c 를 다시 쓰면 안 됨. 그냥 switch 만

```
git switch feature/data-collection  
# 또는  
# git switch feature/churn-model
```

4-3. 현재 내가 어느 브랜치에 있는지 확인

```
git branch
```

* 표시가 붙은 브랜치가 현재 브랜치임. 예: * feature/data-collection

5. Pull / Add / Commit / Push 사용 방법

이 네 단계를 이해하면 Git 협업의 대부분을 처리할 수 있음. 아래 순서를 기본 루틴으로 사용

```
git switch 내_담당_브랜치
git pull origin 내_담당_브랜치

# 파일 작업

git status
git add .
git commit -m "feat: 작업 내용"
git push origin 내_담당_브랜치
```

5-1. git pull — 원격의 최신 내용 받기

원격 GitHub 브랜치의 최신 commit 을 내 PC 의 현재 브랜치에 반영

```
git pull origin feature/data-collection
```

확인 pull 하기 전에 현재 브랜치가 맞는지 git branch 또는 git status 로 확인

5-2. git status — 변경 파일 확인

```
git status
```

어떤 파일이 수정되었는지, 새로 생겼는지, 아직 commit 되지 않았는지를 보여줌

add/commit 전에 반드시 한 번 확인하는 것이 좋음

5-3. git add — commit 할 파일 선택

현재 변경된 모든 파일을 commit 대상으로 올릴 때:

```
git add .
```

특정 파일만 올릴 때:

```
git add src/collector.py
```

민감한 키, 대용량 원본 데이터, 개인 설정 파일이 포함되지 않았는지 git status 로 다시 확인

5-4. git commit — 변경 내용을 하나의 작업 단위로 저장

```
git commit -m "feat: Match-V5 수집 기능 추가"
```

commit 은 “내 PC 의 Git 기록”에 저장하는 단계입니다. commit 만 했다고 GitHub 에 올라가는 것은 아님.

접두사	언제 사용?	예시
feat	새 기능 추가	feat: 이탈 예측 모델 추가
fix	버그 수정	fix: API 응답 오류 처리
docs	문서 수정	docs: README 실행 방법 추가
refactor	동작은 같고 코드 구조 개선	refactor: 수집 함수 분리
test	테스트 관련	test: 예측 함수 테스트 추가
chore	설정·기타 작업	chore: requirements 업데이트

5-5. git push — 내 commit 을 GitHub 에 올리기

```
git push origin feature/data-collection
```

push 가 완료되면 GitHub 에서 해당 feature 브랜치에 방금 commit 한 내용이 보임

5-6. 가장 안전한 기본 루틴

1) 담당 브랜치로 이동

```
git switch feature/data-collection
```

2) 원격 최신 내용 받기

```
git pull origin feature/data-collection
```

3) 파일 작업 후 변경 확인

```
git status
```

4) commit 대상 등록

```
git add .
```

5) 다시 확인

```
git status
```

6) commit

```
git commit -m "feat: Riot API 데이터 수집 로직 추가"
```

7) GitHub 로 push

```
git push origin feature/data-collection
```

6. 실제 하루 작업 예시

예를 들어 데이터 수집 담당자가 collector.py 를 수정한다고 가정

```
# VSCode 터미널

git switch feature/data-collection
git pull origin feature/data-collection

git status
# collector.py 수정

git status
git add src/collector.py
git commit -m "feat: 최근 20 경기 수집 로직 추가"
git push origin feature/data-collection
```

이 과정까지는 develop 이나 main 에 아무 영향이 없습니다. 자신의 feature 브랜치에만 작업 결과가 반영됨

7. Pull Request 로 develop 에 합치는 방법

기능 단위 작업이 끝나면 GitHub 에서 Pull Request 를 생성

1. 자신의 feature 브랜치에 마지막 push 까지 완료
2. GitHub 저장소 페이지로 이동
3. Pull requests → New pull request 를 선택
4. base 브랜치를 develop 으로 선택
5. compare 브랜치를 자신의 feature 브랜치로 선택
6. 변경 내용을 확인하고 PR 제목/설명을 작성
7. Create pull request 를 누르다
8. 팀원이 확인한 뒤 문제가 없으면 develop 으로 merge

```
base: develop
compare: feature/data-collection
```

PR 설명에는 최소한 “무엇을 바꿨는지 / 어떤 파일이 주요 변경인지 / 실행 또는 확인 방법 / 주의할 점”을 작성

8. develop 최신 내용을 내 feature 에 반영하는 방법

다른 팀원의 기능이 develop 에 합쳐진 뒤 그 최신 내용을 내 feature 에서도 사용해야 할 수 있음

```
# 현재 변경사항을 먼저 commit/push 한 상태에서 진행

git switch develop
git pull origin develop

git switch feature/data-collection
git merge develop

git push origin feature/data-collection
```

충돌이 발생하지 않으면 merge 가 자동 완료됨

충돌이 발생하면 충돌 파일을 열어 어떤 내용을 남길지 정한 뒤 add → commit → push

9. 자주 발생하는 오류와 해결법

상황 / 메시지	원인	해결
fatal: only one reference expected	git switch -c 처럼 -c 를 띄움	git switch -c feature/... 로 붙여서 입력
fatal: a branch named ... already exists	이미 로컬에 같은 브랜치가 있음	git switch feature/... 사용
pathspec ... did not match	브랜치명 오타 또는 원격 정보 미수신	git fetch --all --prune 후 git branch -a 확인
Permission denied (publickey)	SSH 키 인증 실패	ssh -T git@github.com 확인, 공개키 등록/ssh-add 확인
rejected / non-fast-forward	원격에 내가 받지 않은 commit 이 있음	먼저 git pull origin 현재브랜치 후 충돌 해결, 다시 push
nothing to commit	변경 내용이 없거나 이미 commit 됨	git status 로 상태 확인

9-1. git branch -a 에서 화면 아래에 :가 보일 때

오류가 아닙니다. 출력 내용이 pager(less)로 열린 상태 -> 키보드에서 q 를 누르면 빠져나오기

9-2. 수정 중인데 다른 브랜치로 이동이 안 될 때

아직 commit 하지 않은 변경 때문에 브랜치 전환 시 파일이 덮어써질 위험이 있으면 Git 이 이동을 막을 수 있음.

가장 안전한 방법은 현재 작업을 commit 한 뒤 이동하는 것임

```
git status
git add .
git commit -m "wip: 작업 중 내용 임시 저장"
git switch 다른브랜치
```

10. 팀원이 꼭 기억해야 할 체크리스트

시점	확인할 것
최초 1 회	SSH 인증 완료 → SSH URL 로 clone → git fetch → 담당 feature 브랜치 생성/추적
작업 시작 전	git branch 로 위치 확인 → git pull origin 담당브랜치
작업 중	민감정보/API Key 와 대용량 데이터 파일을 저장소에 올리지 않기
commit 전	git status → 필요한 파일만 git add
작업 저장	git commit -m "타입: 내용" → git push origin 담당브랜치
기능 완료	GitHub 에서 feature → develop Pull Request
최종 반영	develop 통합 검증 후 develop → main Pull Request

팀원에게 전달할 가장 짧은 설명

요약 처음에는 SSH 로 저장소를 clone 하고 본인 담당 feature 브랜치로 들어감 -> 매 작업 시작 전에 pull, 작업 후에는 status → add → commit → push 순서로 저장 -> 기능이 끝나면 feature 브랜치에서 develop 으로 PR 을 보내고, main 에는 직접 작업하지 않음

복붙용 명령어 템플릿

```
# 최초 clone
git clone git@github.com:OWNER/REPOSITORY.git
cd REPOSITORY
git fetch --all --prune

# 최초 1 회: 담당 브랜치 생성/추적
git switch -c feature/브랜치명 --track origin/feature/브랜치명

# 평소 작업
git switch feature/브랜치명
git pull origin feature/브랜치명
git status
git add .
git commit -m "feat: 작업 내용"
git push origin feature/브랜치명

# 기능 완료: GitHub PR
# feature/브랜치명 → develop
```